Institutional-Grade Quantitative Trading System

Knowledge Graph, NLP, & Multi-Agent Consensus Framework

This document serves as a comprehensive technical blueprint for building a highly advanced, automated quantitative trading system. Moving beyond standard sentiment analysis, this architecture utilizes a **Knowledge Graph (KG)** and **Graph Neural Networks (GNN)** to predict market movements by mapping deep supply-chain, regulatory, and competitive relationships.

Part 1: Core System Architecture

To operate efficiently in production without severe execution slippage, the system completely decouples heavy data processing from live execution logic. The architecture consists of five distinct layers:

1. **Data Ingestion Pipeline:** Synchronized streaming of structured price/volume data and unstructured news/social text.
2. **Knowledge Extraction Engine:** Natural Language Processing (NLP) models that extract entities and deduce market relationships.
3. **Graph Storage Layer:** A native graph database designed for rapid multi-hop traversal.
4. **Graph Machine Learning Engine:** A Graph Attention Network (GAT) that processes market telemetry against structural node relationships.
5. **Strategy & Execution Engine:** The localized loop that manages portfolio risk and interfaces directly with exchange APIs.

Part 2: Step-by-Step Implementation Guide

Step 1: The Dual Data Ingestion Pipeline

You must establish two parallel data streams synced by exact timestamps:

- **Structured Stream:** APIs like Alpaca, Binance, or Polygon.io providing Tick-level data (Price, Volume) and derived technical indicators (RSI, MACD, Bollinger Bands).

- **Unstructured Stream:** RSS feeds, Reuters/Bloomberg scrapers, and SEC filing APIs to ingest live corporate announcements.

Step 2: Knowledge Extraction (NLP to Graph)

Raw text must be converted into structured Nodes (Entities) and Edges (Relationships). Utilize a specialized financial transformer like *FinBERT* alongside spaCy to perform Named Entity Recognition (NER).

- **Node Types:** *Company, Product, Macro_Event*
- **Edge Types:** *SUPPLIES, COMPETES_WITH, REGULATES, IMPACTS*

Entity resolution is critical here: map variations like "Apple Inc.", "Apple", and "$AAPL" to a single unique Node ID in your system.

Step 3: Graph Storage Layer (Neo4j)

Do not use SQL. Institutional systems require rapid traversal of complex supply webs. Use **Neo4j** or AWS Neptune. Nodes store dynamic attributes (e.g., current RSI), while edges store relationship confidence and temporal weights.

```
// Example Cypher query injecting NLP-discovered market linkage
MERGE (c1:Company {ticker: "NVDA"})
MERGE (c2:Company {ticker: "TSM"})
MERGE (c1)-[r:DEPENDS_ON {type: "Manufacturing"}]->(c2)
SET r.confidence_score = 0.95
```

Step 4: The Graph Neural Network (GNN) Engine

Traditional time-series models treat stocks in isolation. A GNN uses *Message Passing*: a stock node aggregates signals from its neighbors (suppliers, competitors). Build this using **PyTorch Geometric (PyG)**.

Use a **Heterogeneous Graph Transformer (HGT)** to allow the model to dynamically weight the importance of different connection types (e.g., ignoring a minor partnership, but heavily weighting a primary lithium supplier during an EV battery shortage).

Step 5: Strategy and Automated Execution

The GNN outputs a predictive signal. The execution engine evaluates this against strict risk parameters. Crucially, the execution logic must be local to minimize latency, connecting via WebSockets.

> **Critical Safety Rule:** Never deploy an automated system without hard-coded, server-side Stop-Loss (SL) and Take-Profit (TP) limits. The system must autonomously liquidate if the trade moves against the predicted vector by a predefined threshold.

Part 3: Daily Backtrack & Continual Learning Loop

To ensure the system learns from its mistakes, you must implement a nightly post-mortem audit. When a trade hits a stop-loss, it indicates a blind spot in either graph topology or model attention.

```
[Closed at Loss] → [Extract Live News] → [LLM Causal Discovery] →
                        [Heal Graph Topology]
```

1. Unsupervised Text Clustering

Extract all global news published during the failed trade window. Use an algorithm like ***HDBSCAN*** over sentence embeddings to cluster unexpected macro events that your graph missed.

2. Causal Link Discovery

Pass the anomaly news cluster to an LLM agent with the prompt: *"Stock XYZ crashed today. Based on this text cluster, deduce the missing economic relationship affecting this asset."*

3. Dynamic Loss Optimization (Contrastive Learning)

Penalize the GNN for missing the causal nodes. Run a daily mini-batch update adjusting the loss function to heavily weight the edges that were ignored during the crash:

$$L_{backtrack} = L_{prediction} + \lambda \Sigma (1 - \alpha_{ij})^2$$

Where α_{ij} is the attention weight the model incorrectly gave to the critical risk node.

Part 4: Multi-Agent Consensus Network

Running a single AI model exposes the system to confirmation bias. Implement an actor-critic LLM layer to debate trades before execution.

- **Momentum Bull Agent:** Scans the graph for expansion patterns, positive velocity, and bullish momentum.
- **Risk Bear Agent:** Searches the graph for hidden liabilities, supply chain choke-points, and overbought indicators.
- **Synthesis Critic:** Audits both arguments against the historical Daily Backtrack logs to prevent repeating past mistakes.

The system calculates a final execution probability threshold based on the debate:

$$P(\text{Trade}) = w_1 S_{\text{Bull}} - w_2 S_{\text{Bear}} - C_{\text{Risk}}$$

If the final $P(\text{Trade})$ exceeds the required threshold, the order is passed to the broker API.

Part 5: Production Safeguards & Optimization

1. Temporal Graph Decay

Market relationships expire. A supplier contract from 2024 should not carry the same weight in 2026. Transition to a **Temporal Graph Network (TGN)** where edge weights decay over time unless validated by recent news.

$$w_{ij}(t) = w_{ij}(t_0) \times e^{-\gamma (t - t_0)}$$

Where γ is a domain-specific decay constant, forcing the graph to prune obsolete structural noise.

2. HMM Regime Circuit Breaker

Micro-alpha strategies fail during macro-liquidity crashes (Black Swan events). Deploy a **Hidden Markov Model (HMM)** that monitors global volatility (VIX, yield curves) to classify the market state. If the HMM detects a "High-Stress Turbulent" regime, it automatically overrides the system to slash risk exposure limits by 70% and gives the Bear Agent absolute veto power.

Document compiled for offline review and strategic system implementation. Focus initially on standing up the Neo4j environment and data pipelines before training the neural networks.